

ALGORITMOS E PENSAMENTO COMPUTACIONAL



- Introdução à Linguagem C

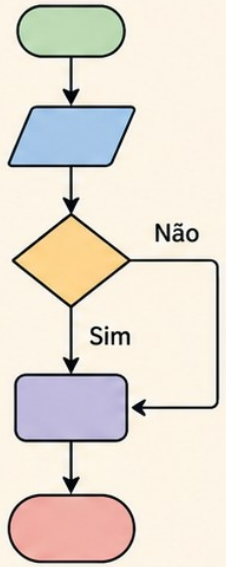
Prof. Marco Antonio

2º semestre 2026

Reflexão



> PRIMEIRO, RESOLVA O PROBLEMA.
>
> DEPOIS, ESCREVA O CÓDIGO.
>
> John Johnson

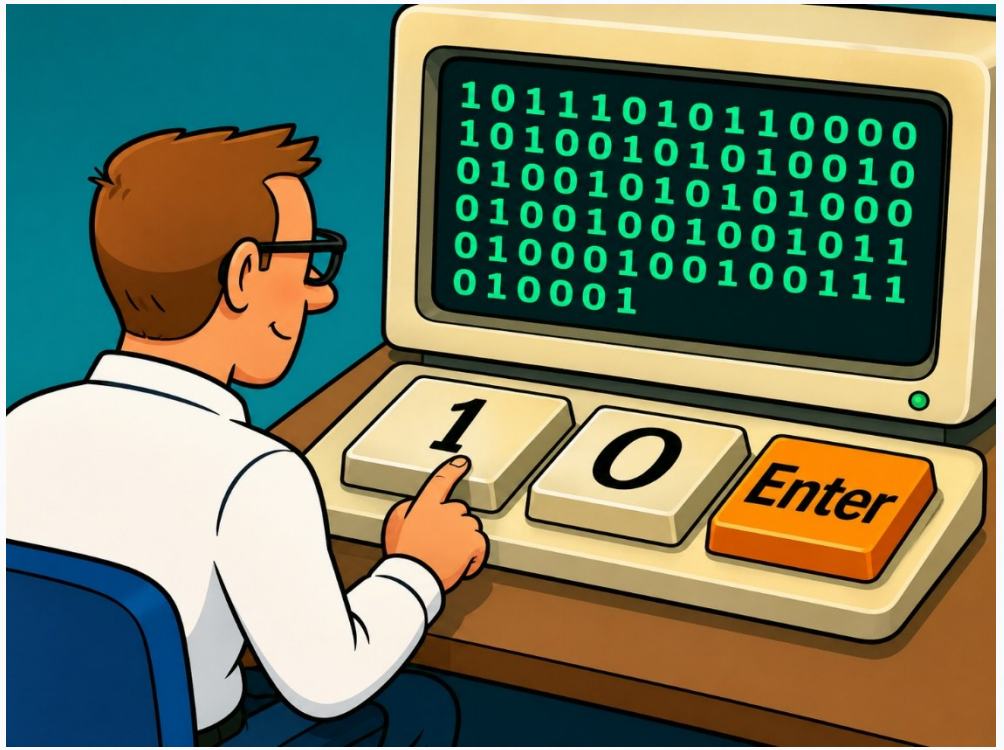


O que é um Programa de Computador?

São instruções escritas em uma linguagem que permite a comunicação entre o programador e o computador (0´s e 1´s) → Linguagem de programação

```
00000000000000000000000000000000
00000000000000000000000100001000
00000010101100011110001100000011
11111000110101001111001111001111
00011010000110001111010101111000
11100000111111000011111101010111
11100000111111000011111101010111
```

Código de máquina



Um arquivo contendo instruções em linguagem de máquina é chamado de **executável**.

Linguagens de programação

Ferramentas para comunicação entre pessoas e computadores

Uma linguagem natural ou idioma (português, inglês, espanhol, etc.) estabelece um vocabulário ou dicionário válido e um grupo de regras sintáticas e semânticas para que as pessoas se expressem adequadamente nessa linguagem e se comuniquem.

De forma semelhante, uma linguagem de programação estabelece um conjunto de regras sintáticas e semânticas, para escrever códigos válidos, a fim de que os computadores possam executar uma determinada função.

Exemplos de Linguagens de Programação



Cada linguagem possui suas próprias palavras (vocabulário) e regras de uso, o que permite que o código seja compreendido pelo computador.

Em resumo: Linguagens naturais conectam pessoas. Linguagens de programação conectar pessoas e computadores.

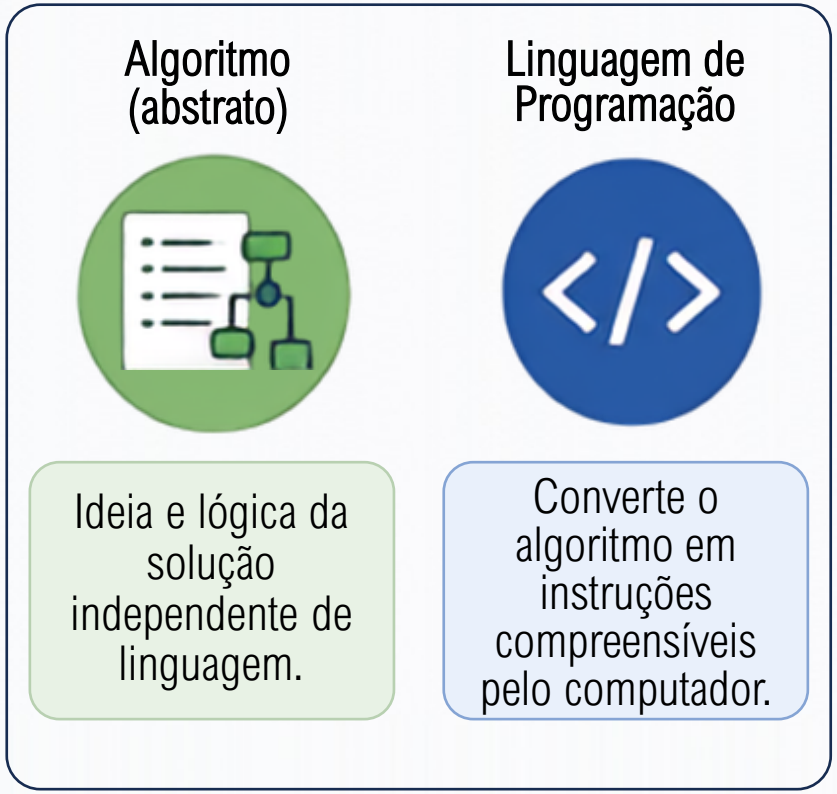
Por que usar uma linguagem de programação?

As linguagens de programação constituem o mecanismo fundamental através do qual os algoritmos abstratos são convertidos em instruções operacionais interpretáveis pelos sistemas computacionais:

A linguagem de programação é o meio pelo qual **transformamos algoritmos em instruções compreensíveis** pelo computador, permitindo a execução real da solução;

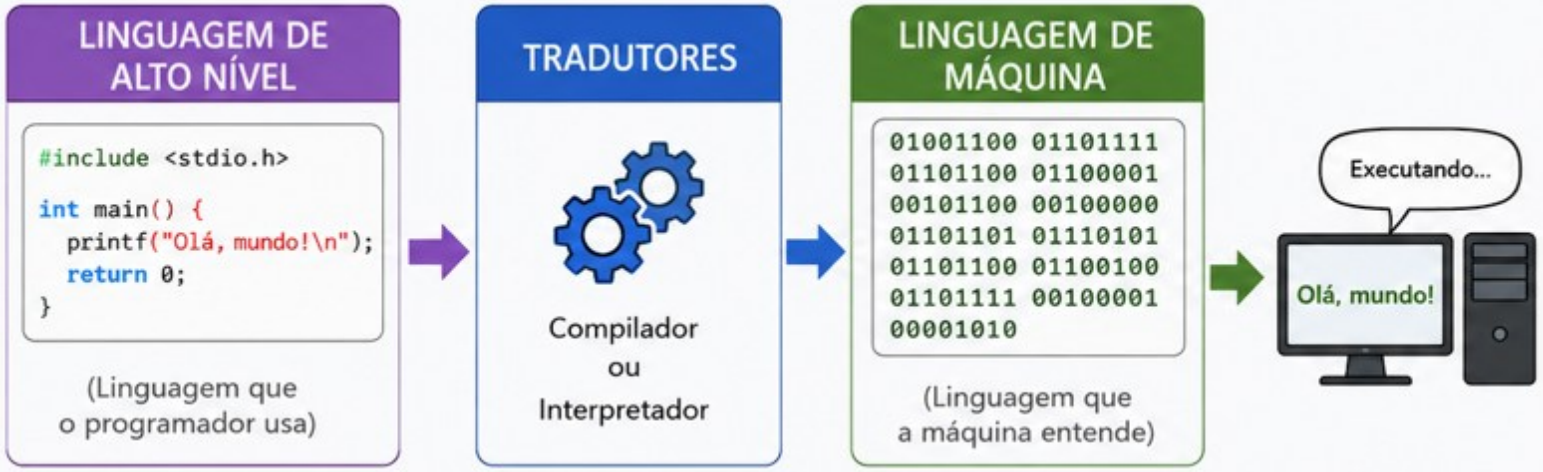
Programar é **dar vida ao algoritmo**, transformando algo escrito em linguagem natural, fluxograma ou pseudocódigo, em instruções que o computador compreende;

É transformar **raciocínio lógico em ação automatizada.**



Como a máquina entende os códigos?

Para que o computador “entenda” um programa é necessário um meio de tradução entre a linguagem de alto nível utilizada no programa e a linguagem de máquina.



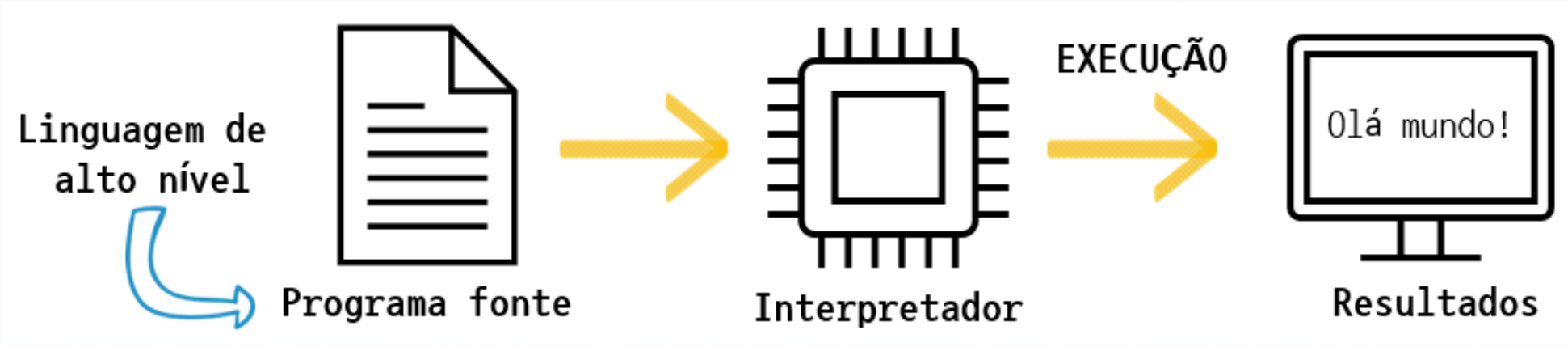


Para essa tarefa temos basicamente dois métodos:

- **Compilador:** traduz o programa escrito em uma linguagem de programação para um programa executável.
- **Interpretador:** traduz e executa o programa linha por linha

Interpretador

- Traduz e faz a checagem da sintaxe e envia para execução, instrução por instrução.
- Precisa estar presente todas as vezes que vamos executar o programa e o processo acima é repetido.

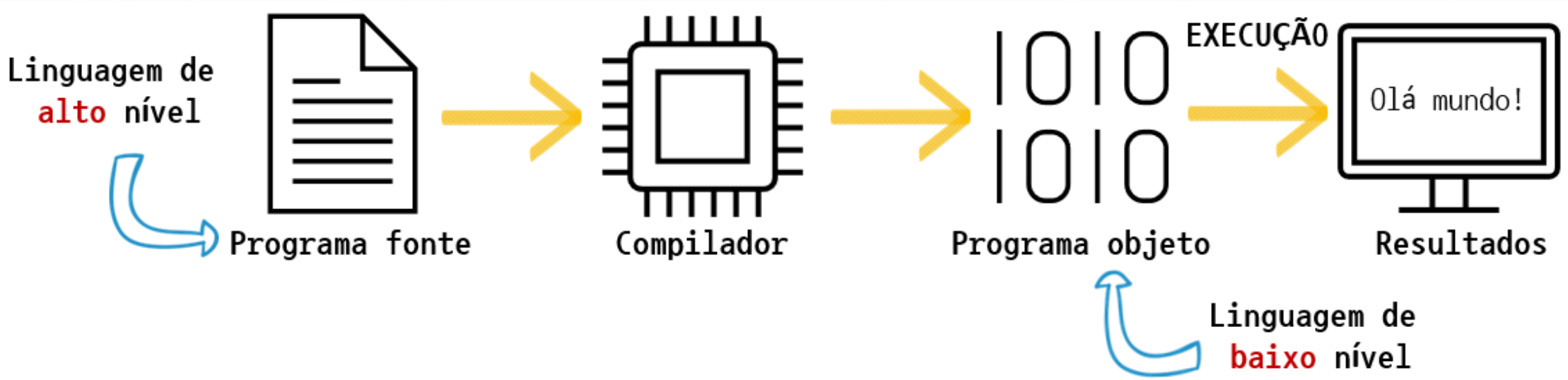


Vantagem: consome menos memória
Desvantagem: execução mais lenta

Exemplo: Python, Ruby, etc

Compilador

Traduz o programa escrito em uma linguagem de programação para um programa equivalente escrito em linguagem de máquina (programa-objeto).



Vantagens:

- Velocidade de execução
- Oculta o código fonte

Exemplos: C, C++, etc.

Desvantagem:

- A cada alteração no programa fonte é necessário gerar novamente o programa-objeto

Os diferentes tipos de linguagens e suas categorias

1. NÍVEL DE ABSTRAÇÃO

Indica o quanto a linguagem está próxima do funcionamento físico do computador.



Baixo Nível

Mais próxima do hardware. Difícil de escrever e manter. Ex.: Linguagem de Máquina, Assembly

Médio Nível

Equilíbrio entre hardware e simplicidade. *Ex.: C*

Alto Nível

Mais próxima da linguagem humana. Mais fácil de escrever e manter. *Ex.: Python, Java, JavaScript*

2. PARADIGMAS DE PROGRAMAÇÃO

É a forma de pensar na hora de escrever um programa. Cada paradigma organiza a lógica do código de um jeito diferente.

Imperativo

Foca em comandos que alteram o estado do programa. *Ex.: C, Pascal*

Funcional

Baseado em funções e expressões. Evita alterar estados. *Ex.: Haskell, Lisp, Scala*

Orientado a Objetos

Organiza o código em objetos que possuem dados e comportamentos. *Ex.: Java, C++, Python*

Declarativo

Foca no que deve ser feito, não em como fazer. *Ex.: SQL, Prolog*

Lógico

Baseado em lógica e regras. Ex.: Prolog

3. SISTEMA DE TIPAGEM

Define como a linguagem trata os tipos de dados e como verifica os valores usados no programa.

Tipagem Estática

Tipos são verificados em tempo de compilação. *Ex.: C, Java, Go*

Tipagem Dinâmica

Tipos são verificados em tempo de execução. *Ex.: Python, JavaScript, PHP*

Tipagem Forte

Não permite conversões implícitas entre tipos incompatíveis. *Ex.: C, Java, Python*

Tipagem Fraca

Permite conversões implícitas entre tipos. *Ex.: C (em partes), JavaScript, PHP*

Ambientes para desenvolvimento de programas (IDE)

São ferramentas que facilitam que fornecem recursos para edição, compilação e execução de programas e outros recursos importantes.



NetBeans e Eclipse

Principais IDE para desenvolvimento na plataforma Java. Ambas podem ser utilizadas para outras linguagens, como o C.

- Editor de código avançado
- Depurador integrado
- Suporte a plugins
- Gerenciamento de projetos



Microsoft Visual Studio Code

Editor de código-fonte gratuito, leve e extremamente popular desenvolvido pela Microsoft, compatível com Windows, macOS e Linux, além de possuir uma versão executável diretamente no navegador web.

- Leve e rápido
- IntelliSense e autocompletar
- Integração com Git e GitHub
- Extensões ilimitadas
- Execução no navegador



Dev++ e Code::Blocks

IDE clássico, gratuito e de código aberto, amplamente utilizado por iniciantes e estudantes para programar nas linguagens C e C++.

- Simples e fácil de usar
- Compilação com diversos compiladores
- Execução com um clique
- Gerenciamento de projetos
- Leve e eficiente

Pseudocódigo: a ponte entre a ideia e o código



O que é?

O pseudocódigo é a descrição do que o algoritmo faz passo a passo em um **português estruturado.**



Por que usar?

É o mais fácil de fazer e pode ser feito mentalmente e em qualquer lugar em que você estiver.



Para que serve?

Trata-se de uma forma excelente para iniciar o aprendizado de lógica e é extremamente acessível.



Como se relaciona com o código?

Ele ajuda a planejar a solução antes de escrevê-la em uma linguagem de programação (como C), reduzindo erros e retrabalho.

Exemplo de pseudocódigo

```
1  INÍCIO
2    DECLARE raio, area, perimetro : REAL
3    ESCREVA "Informe o raio do círculo (cm): "
4    LEIA raio
5    perimetro ← 2 * 3.14159 * raio
6    area ← 3.14159 * raio * raio
7    ESCREVA "Perímetro: ", perimetro
8    ESCREVA "Área: ", area
9  FIM
```

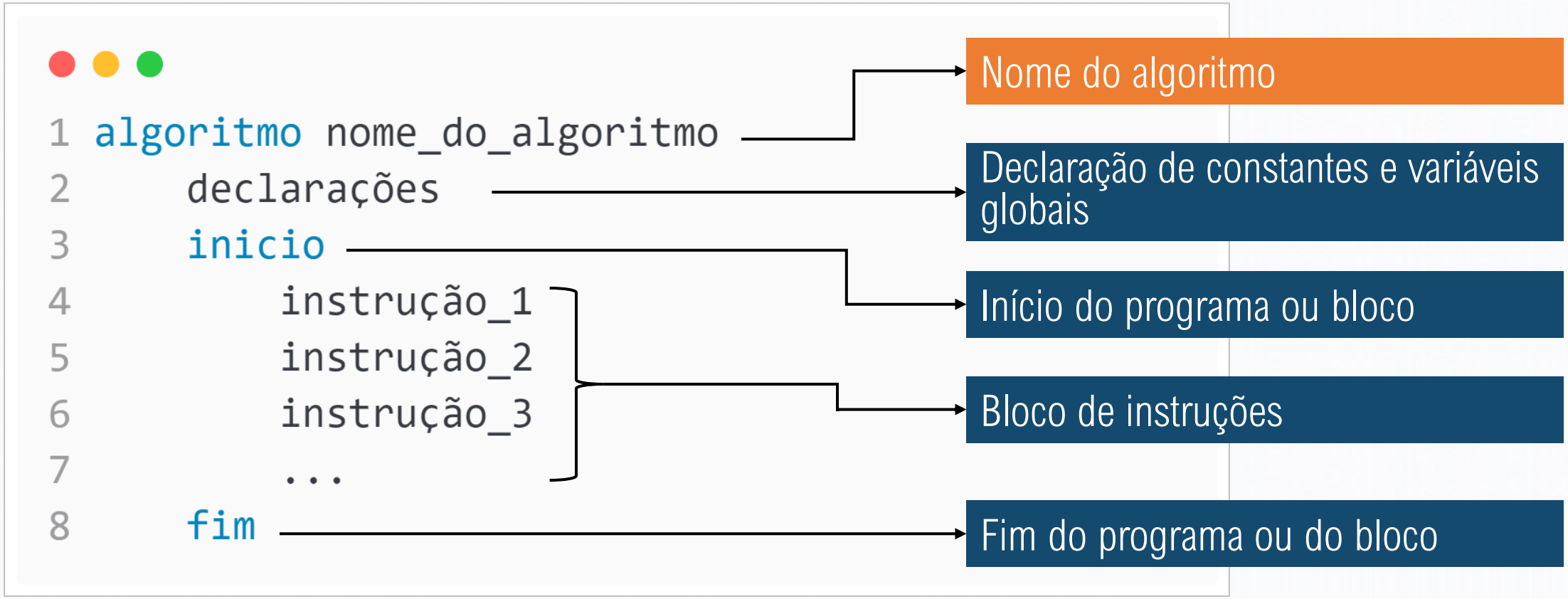
Convenções comuns

`INÍCIO / FIM`	– marcam o início e o fim do algoritmo
`DECLARE`	– declara variáveis
`LEIA`	– entrada de dados
`ESCREVA`	– saída de dados
`←`	– atribuição (recebe o valor da direita)

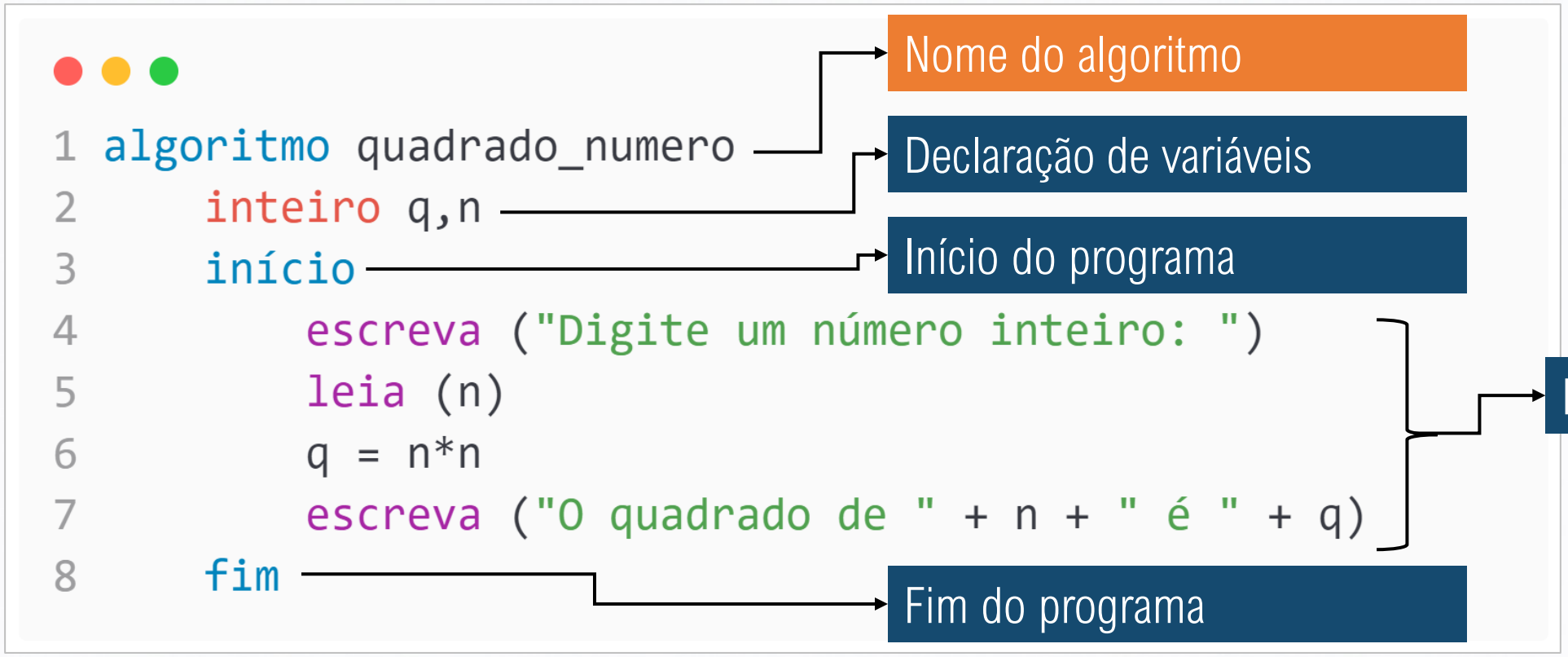
Dica: Antes de começar a programar, escreva o algoritmo! Ele será o mapa para a solução do problema.

Notação utilizada na aula... PSEUDOCÓDIGO

Nesta disciplina, adotaremos a seguinte estrutura para o pseudocódigo:



Exemplos de algoritmo (pseudocódigo)



Algoritmo que calcula e exibe o quadrado de um número digitado pelo usuário.

PseudoCódigo

Exemplos de algoritmo (pseudocódigo)

```
algoritmo média
    real: media, n1, n2
    início
        escreva("Digite o primeiro número: ")
        leia(n1)
        escreva("Digite o segundo número: ")
        leia(n2)
        media = (n1 + n2) / 2
        escreva("A média dos números é: " + media)
    fim

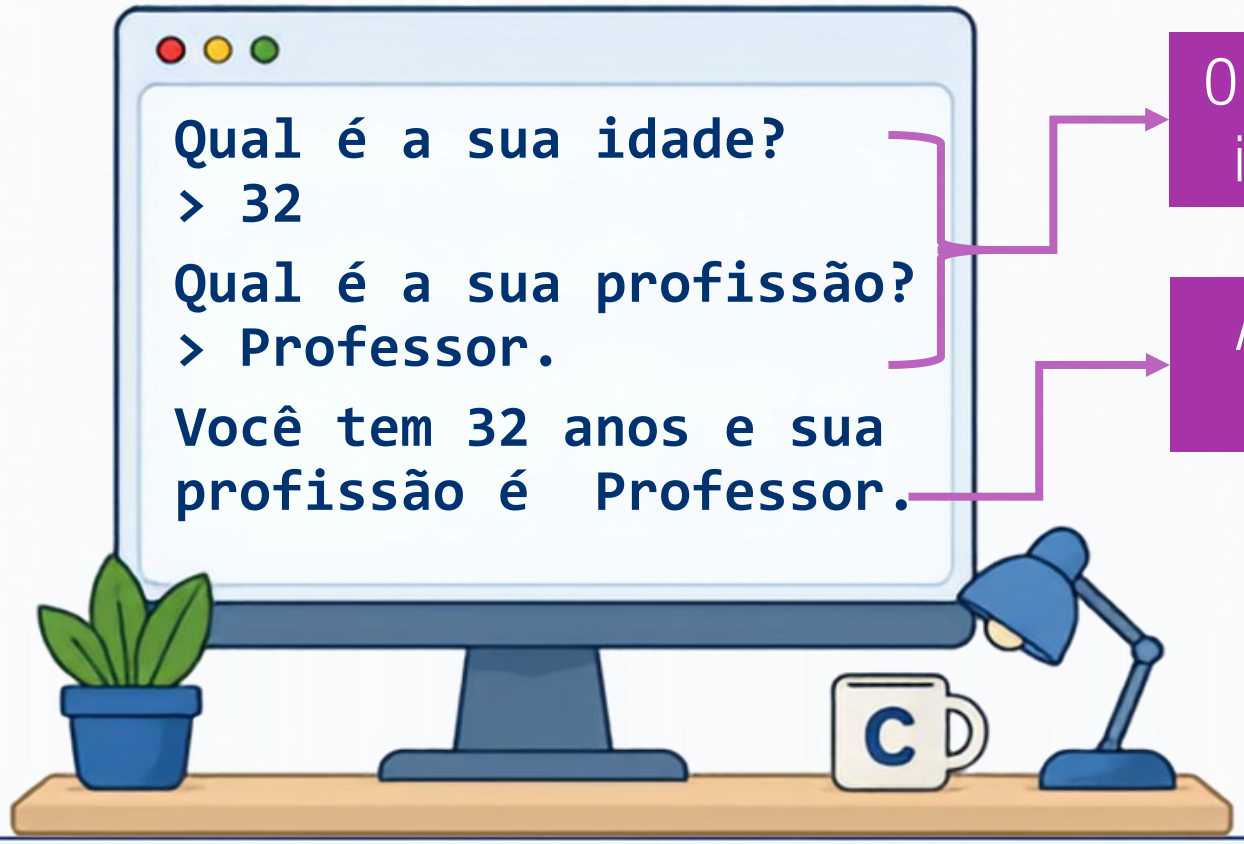
```



Algoritmo que calcula e exibe a média entre dois números digitados pelo usuário.

Exemplo de aplicação

Problema: Crie um algoritmo que solicite ao usuário a sua idade e armazene esse dado em uma variável, solicite também a profissão e armazene em outra variável. Após obter os dados, apresente em uma única mensagem os valores digitados.



O programa aguarda o usuário digitar as informações e armazena nas variáveis.

As informações digitadas pelo usuário são exibidas na tela.

Por que usar pseudocódigo?

Exemplos de aplicação

// Exemplo 1: idade e profissão do usuário em Pseudocódigo

```
algoritmo Exemplo1
    inteiro idade
    literal profissao
    início
        escreva("Digite a sua idade: ")
        leia(idade)
        escreva("Digite a sua profissão: ")
        leia(profissao)
        escreva("Profissão: " + profissao)
        escreva("Idade: " + idade)
    fim
```



Linguagem C

Nesta disciplina utilizaremos a **linguagem C** para implementação de **alguns algoritmos**.

A linguagem de programação C foi desenvolvida no início dos anos 1970, motivada pela necessidade de uma linguagem **mais poderosa e flexível** para o desenvolvimento do sistema operacional **UNIX**.

C é uma linguagem **estruturada**, o que facilita o aprendizado dos principais conceitos da programação, como variáveis, funções, estruturas de controle e operadores.

Permite um **acesso mais direto** aos recursos do computador, como memória e dispositivos, sendo ideal para quem deseja entender o que realmente acontece "por trás do código".

Características da linguagem C

- Linguagem rápida e eficiente
- Portável entre diferentes sistemas
- Estruturada e modular
- Base para muitas outras linguagens

exemplo.c

```c
#include <stdio.h>

int main() {
    int idade;
    printf("Digite sua idade: ");
    scanf("%d", &idade);

    printf("Sua idade é %d anos.\n", idade);

    return 0;
}
```

C continua sendo amplamente utilizada em **sistemas operacionais**, **aplicativos de alto desempenho**, **drivers**, **compiladores**, **embarcados** e muito mais!

Linguagem C: aplicações no mundo real

A linguagem C é amplamente utilizada em:

- Sistemas operacionais (como o Linux);
- Sistemas embarcados;
- Automação industrial e de processos;
- Compiladores e Interpretadores.



Base para outras linguagens:

Aprender C ajuda a formar uma base sólida para transitar com facilidade por outras linguagens modernas, como **C++, Java, Python** e até mesmo linguagens específicas de automação.

Estrutura básica de um programa em C

Inclui arquivos de cabeçalho necessários ao programa

Permite usar comandos de entrada e saída, como escrever mensagens na tela (stdio.h)

```c
#include <stdio.h>

int main(){
    printf("Olá mundo\n");
    return 0;
}
```

Bloco de instruções

Função responsável pelo início da execução do programa (main)



A maioria dos programas iniciais em C terá uma estrutura semelhante a essa.

Estrutura básica de um programa em C

No início do programa, normalmente colocamos **bibliotecas, constantes** e **outras declarações** que poderão ser usadas ao longo do código.

Todo programa em C deve conter uma função **`main()`**, responsável pelo início da execução do programa, onde colocamos os comandos que queremos executar.

Usamos **`int main(void)`** porque a função devolve um número inteiro ao sistema operacional. O comando **`return 0`** indica que o programa terminou corretamente.

As chaves **`{}`** delimitam o bloco de instruções que será executado pela função.

A maioria das instruções em C termina com **`;`** (ponto e vírgula).

Exemplos C - exemplo_inicial.c

```c
#include <stdio.h>
#define PI 3.14159

int main() {
    float raio, area;
    printf("Digite o raio: ");
    scanf("%f", &raio);
    area = PI * raio * raio;
    printf("Área = %.2f\n", area);
    return 0;
}
```

Declarações
Bibliotecas, constantes e outras declarações globais.

Função principal
Ponto de entrada do programa.

Bloco de instruções
Comandos executados dentro da função main().

Finalização
return 0 indica término correto do programa.

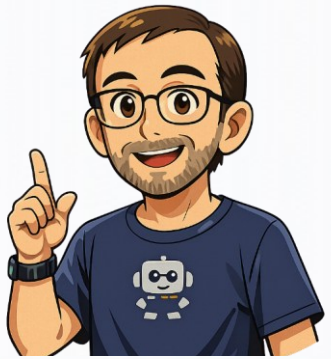
Fim do bloco
As chaves delimitam o bloco da função.

Em C, clareza e organização do código são essenciais! Cada parte do programa tem um papel importante.

Tipos de Dados

Quando criamos um algoritmo, é necessário identificar:

- os **dados** que serão processados;
- as **instruções** que determinarão como esses dados serão manipulados.



Os dados representam as informações utilizadas pelo algoritmo, como números, textos, caracteres ou valores lógicos.

Neste primeiro momento, vamos classificar os dados de forma geral. Depois, veremos como eles são representados na linguagem C.

Exemplos no dia a dia

18	Idade, quantidade de itens, temperatura.
	Nome, endereço, mensagem.
A	Letra de um código, inicial do nome.
	Situação: aprovado/reprovado, ligado/desligado.
	Lista de alunos, cadastro de produtos, arquivo de dados.

Reconhecer os diferentes tipos de dados é o primeiro passo para desenvolver algoritmos corretos e eficientes.

Tipos de dados

Categoria	O que representa	Exemplos	Descrição
123 Numéricos	Quantidades e valores numéricos.	-10, 0, 42, 3.14	Podem ser inteiros ou reais (com casas decimais).
A Textuais	Sequências de caracteres (textos).	"Olá", "C", "Algoritmo"	Utilizados para armazenar palavras, frases ou textos.
Caracteres	Um único símbolo (letra, número, símbolo).	"A", "9", "*", "@"	Armazenam apenas um caractere por vez.
Lógicos	Valores de verdade: verdadeiro ou falso.	VERDADEIRO, FALSO (TRUE, FALSE)	Usados em testes e decisões do algoritmo.
1010 1010 Outros (especiais)	Estruturas mais complexas, coleções ou combinações de dados.	vetores, registros, estruturas, arquivos	Usados para representar conjuntos e dados mais elaborados.

se Liga Aí

Em algoritmos, o separador decimal pode variar conforme a notação adotada. Em C, utiliza-se o ponto decimal.

Tipos de dados primitivos em C

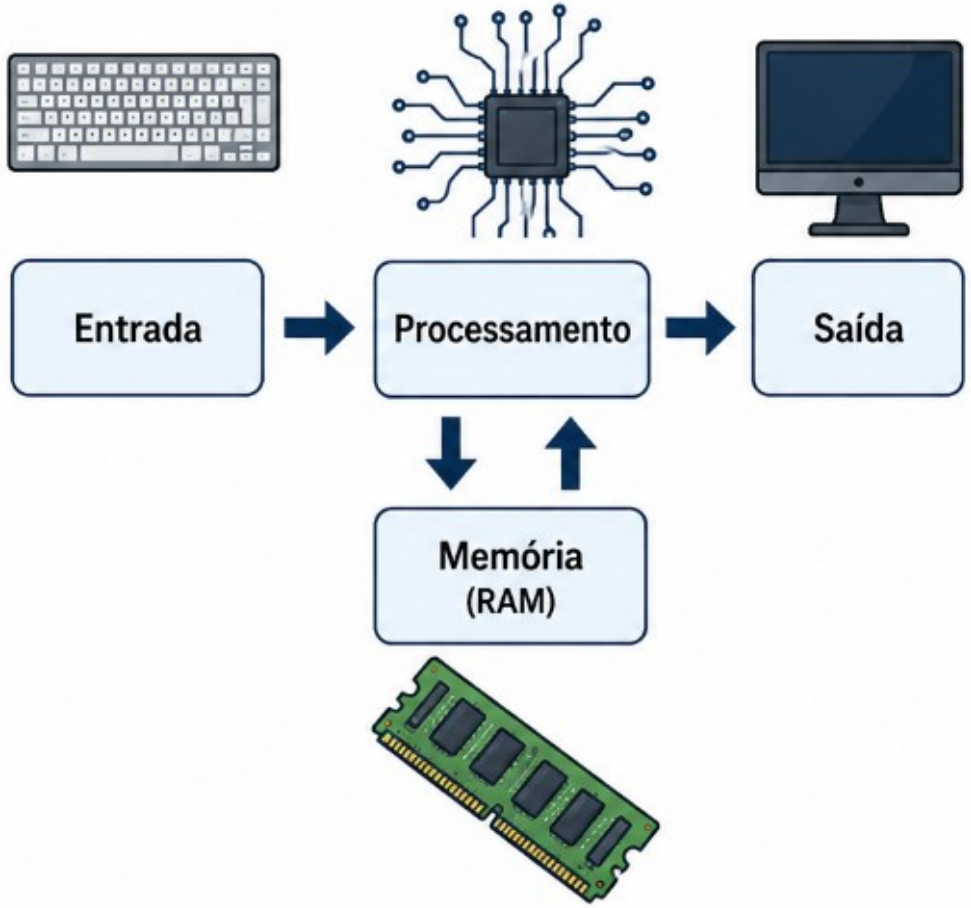
Classificação geral	Em C	Exemplo
123 inteiro	`int`	`int idade = 20;`
1.3 real	`float / double`	`float altura = 1.75;`
A caractere	`char`	`char conceito = 'A';`
lógico	`int ou bool`	`int aprovado = 1;`



- Caractere em C usa aspas simples: `'A'`;
- Texto em C usa aspas duplas: `"Maria"`, porém, os textos em C não são um tipo simples como `String` em Java ou Python. Textos completos serão tratados depois como sequências de caracteres (`Strings`).
- Em algoritmos, o separador decimal pode variar conforme a notação adotada. Na linguagem C, utilizamos o ponto decimal.

Processamento de dados

Modelo abstrato



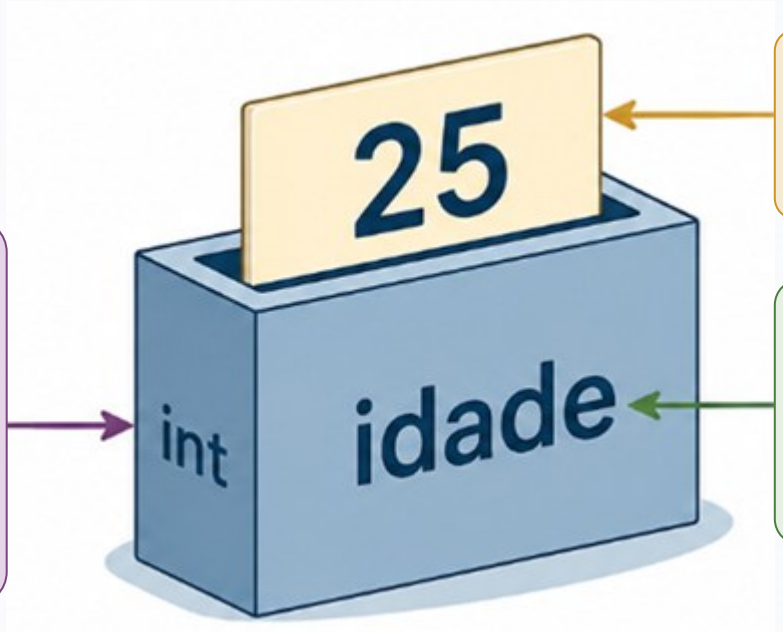
Basicamente, um programa manipula dados que são, em geral, **armazenados em variáveis** localizadas na memória RAM.

- A variável modela a unidade de memória;
- Possui um identificador;
- Associa um endereço físico de memória;
- Tem um tamanho em bytes.

O que são variáveis em programação?

Uma **variável** é um **espaço de memória usado para armazenar temporariamente um valor**.

`Tipo de dado`
Define o conjunto de dados que a variável pode armazenar e quantidade de memória que ela ocupa.



`Conteúdo (dado)`
Valor armazenado na variável.

`Nome (identificador)`
Nome que identifica a variável no programa.

```c
// Exemplo em C
int idade = 25;
```

tipo nome conteúdo

```c
// Cria uma variável chamada 'idade'
// do tipo inteiro (int) e armazena
// o valor 25 nela.
```

Identificador (nome da variável)

> É através dele que você irá se referir a variável no código para lhe atribuir algum dado ou para recuperar um dado que foi armazenado.

O identificador de uma variável **deve ser representativo do seu conteúdo** e seguir as seguintes regras:

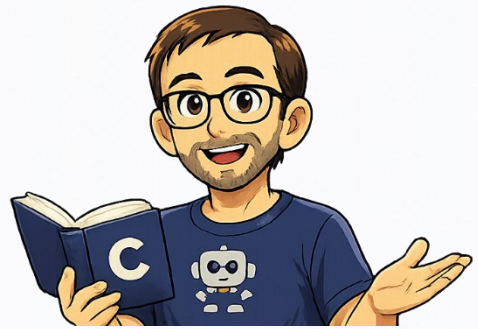
1. não pode começar com número;
2. não pode conter espaço;
3. não pode conter acentos;
4. não pode conter símbolos como @, #, -, !;
5. não pode ser uma palavra reservada da linguagem.

Exemplos:

Válidos	Inválidos
`idade`	`1idade`
`nota1`	`nota final`
`media_final`	`média`
`quantidade`	`float`

Declaração de variáveis

- Toda variável utilizada na elaboração de um programa precisa ser definida antes de serem utilizadas. Isto se faz necessário para permitir que o compilador reserve um espaço na memória para as mesmas.
- Para isso, devemos definir o identificador da variável e o seu tipo por meio do que chamamos de declaração de variáveis.
- Sua forma geral é a seguinte:



Exemplos - teste.c

```
1 // declaração de variável
2 tipo_de_dado variavel;
3 //ou
4 tipo_de_dado variavel = valor_inicial;
```

A atribuição de um valor inicial é opcional e depende da lógica.

Declaração de variáveis

Exemplos

Pseudocódio

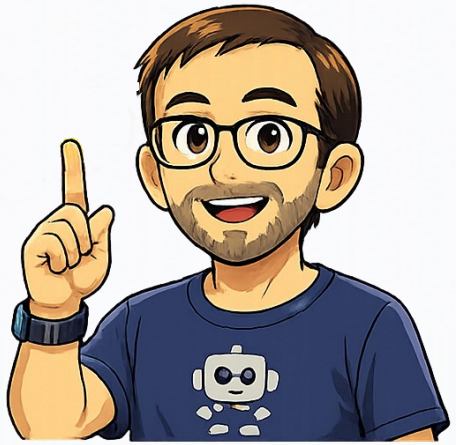
```
inteiro idage, num1
real nota1, media
literal nome
logico aprovado
```

C

```
int idage;
float nota1, nota2,
media;
char conceito;
```

Pseudocódigo: representação da lógica de forma simples e estruturada.

C: implementação da lógica utilizando a sintaxe da linguagem C.



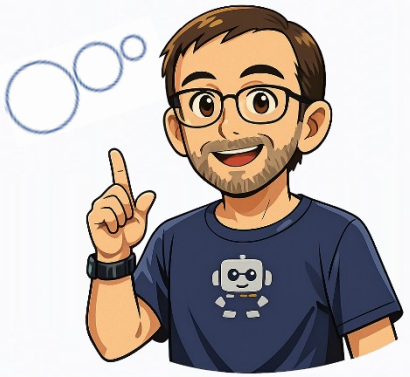
Inicialização de variáveis

Existem várias maneiras de atribuir valores a variáveis:

- Dizendo no algoritmo qual o valor a variável deve assumir:

```
1 algoritmo compras
2     real preco
3     início
4         preco = 12.99
5     fim
```

Neste exemplo, a variável preco recebe o valor 12.99



Inicialização de variáveis

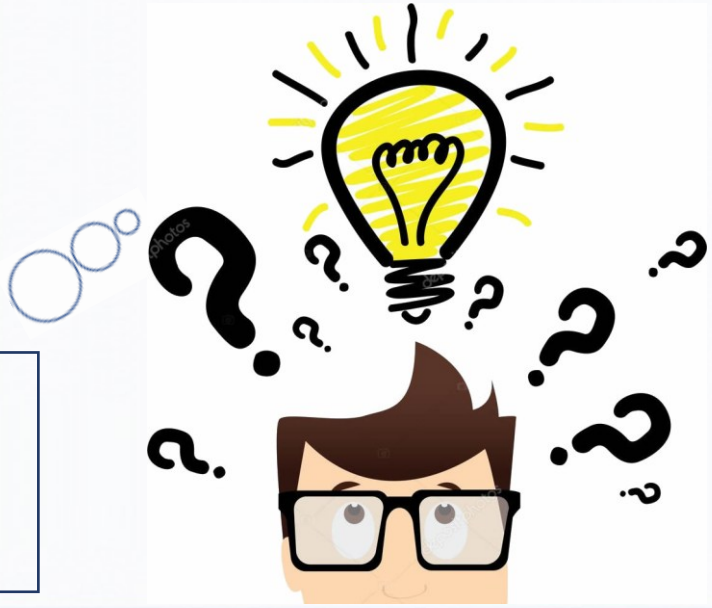
Existem várias maneiras de atribuir valores a variáveis:

- Definindo que uma variável assume o valor de uma outra variável:

```
1 algoritmo inteiros
2     inteiro n1, n2
3     início
4         n1 = 10
5         n2 = n1
6     fim
```



Qual o valor está armazenado na variável n2 após a execução deste algoritmo?????



Inicialização de variáveis

Existem várias maneiras de atribuir valores a variáveis:

- Atribuindo a uma variável o resultado de uma expressão:

```
1 algoritmo atribuicao
2     real a, b, c
3     início
4         a = 12.05
5         b = 5.20
6         c = a * b
7     fim
```

- Usuário digitando o valor utilizando um comando de entrada, como veremos a seguir.

Comandos de Entrada e Saída (Input/Output)

Comando de saída em Pseudocódigo:

escreva cuja finalidade é exibir uma mensagem, que pode ser um texto, o conteúdo de uma variável, ou ambos juntos!

Exemplo:

```
algoritmo soma_simples
    inteiro num
    inicio
        escreva("Digite um número")
        leia(num)
        escreva("O número digitado foi " + num)
fim
```

Somente mensagem

Comando de saída

Mensagem + variável



Comandos de saída em C

- Em C, usamos a biblioteca **`stdio.h`** para entrada e saída padrão, permitindo ler dados do teclado e escrever na tela.
- A função **`printf()`** é responsável por escrever mensagens e valores na tela. Sua forma geral usa um texto de formato e, quando necessário, uma lista de variáveis.

Exemplos - ola_mundo.c

```c
#include <stdio.h>

int main(){
    printf("Olá mundo\n");
    return 0;
}
```

Escreve uma mensagem na tela

Comando de saída em C

- Em C, usamos a biblioteca **`stdio.h`** para entrada e saída padrão, permitindo ler dados do teclado e escrever na tela.
- A função **`printf()`** é responsável por escrever mensagens e valores na tela. Sua forma geral usa um texto de formato e, quando necessário, uma lista de variáveis.

Exemplos - ola_mundo.c

```c
#include <stdio.h>

int main(){
    int idade = 18;
    printf("Você tem %d anos de idade.\n", idade);
    return 0;
}

```

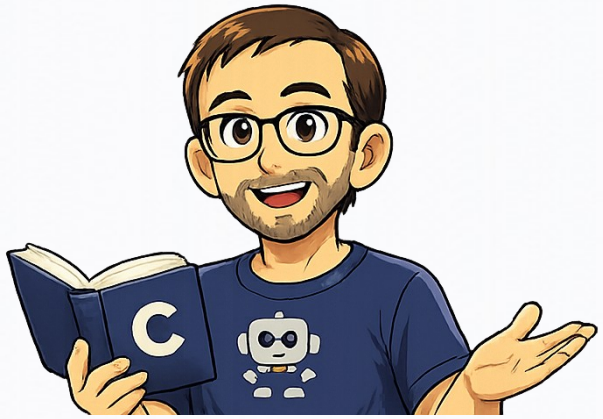
Caractere de escape que cria uma nova linha

Escreve um texto e o conteúdo de uma variável na tela

Comando de saída em C

Especificadores de formatos mais utilizados:

Formato	Descrição
%d	inteiro (**int**)
%c	caractere (**char**)
%s	cadeira de caracteres (**texto / String**)
%f	número decimal de precisão simples (**float**)
%lf	número decimal de precisão dupla (**double**)
%.2f	número com duas casas decimais (**float**)



! O tipo **double** é usado para armazenar números decimais com alta precisão (8 bytes), sendo o padrão para operações matemáticas que exigem muita precisão.

Exemplos de aplicação

1- Faça um programa em C que exiba os dados de um aluno, incluindo RGM e media.

Exemplos C - exemplo1.c

```c
#include <stdio.h>

int main() {
    int rgm = 12345;
    float media = 8.758;
    printf("Dados do aluno:\n");
    printf("RGM: %d\n", rgm);
    printf("Média: %.2f\n", media);
    return 0;
}
```



Resolvendo problemas de acentuação em C

O problema ocorre quando o programa, o arquivo-fonte e o console usam configurações diferentes para representar caracteres acentuados.

O PROBLEMA

Ao escrever programas em C, palavras com acento ou cedilha podem aparecer incorretamente na tela.

```
OlÃ¡
AcentuaÃ§Ã£o
nÃºmero
```



A SOLUÇÃO

Use a biblioteca locale.h e a localidade de caracteres para que programa e console interpretem os acentos corretamente.

- Inclua a biblioteca: `#include <locale.h>`
- Defina a localidade: `setlocale(LC_CTYPE, "");`

```c
#include <stdio.h>
#include <locale.h>

int main(void)
{
    setlocale(LC_CTYPE, "");
    printf("Olá! Acentuação e ç funcionando corretamente.\n");
    return 0;
}
```

Atenção: `LC_CTYPE` ajusta somente os caracteres, sem alterar o formato dos números decimais.

Comando de saída em C

A função **printf()** permite que os campos de exibição sejam formatados. As formatações mais usadas são o preenchimento com zeros à esquerda, para inteiros, e a especificação do número de casas decimais, para reais.

Exemplo 1.6. Formatação de campos com *printf()*.

```
int a = 678;
float b = 12.3456;

printf("\n|%5d|", a);
printf("\n|%06d|", a);
printf("\n|%7.3f|", b);
printf("\n|%7.2f|", b);
```

Observações

- A largura inclui os dígitos, o sinal (se houver) e o ponto decimal.
- Para inteiros, a especificação ".x" é ignorada.

Especificadores utilizados

Especificador	Significado	Aplica-se a	Exemplo
%5d	Campo com largura mínima de 5 posições. Preenchimento com espaços à esquerda.	Inteiros	| 678|
%06d	Campo com largura mínima de 6 posições. Preenchimento com zeros à esquerda.	Inteiros	|000678|
%7.3f	Número real com largura mínima de 7 posições e 3 casas decimais.	Reais	| 12.346|
%7.2f	Número real com largura mínima de 7 posições e 2 casas decimais.	Reais	| 12.35|

Comandos de Entrada e Saída (Input/Output)

Os algoritmos precisam ser “alimentados” com dados provenientes do meio externo para efetuarem as operações e cálculos e é necessário também mostrar os resultados.

- Comando de entrada em Pseudocódigo:

leia tem como finalidade atribuir o dado a ser fornecido à variável identificada.

Exemplo:

```
algoritmo idade
    inteiro idade
    inicio
        leia(idade)
    fim
```

Comando de entrada de dados

Variável que irá armazenar o dado



Comando de entrada em C

- Em C, usamos a biblioteca **`stdio.h`** para entrada e saída padrão, permitindo ler dados do teclado e escrever na tela.
- A função **`scanf()`** lê valores digitados pelo usuário. Para armazenar o valor em uma variável comum, usamos **`&`** antes do nome da variável.

Exemplos C - entrada.c

```c
#include <stdio.h>

int main(){
    int idade;
    printf("Digite sua idade: ");
    scanf("%d", &idade);
    printf("Você tem %d anos de idade.\n", idade);
    return 0;
}
```

Especificador de formato

Variável

Comando de entrada de dados



Exemplos de aplicação



Em uma cafeteria universitária, é comum que os estudantes façam pedidos rápidos durante o intervalo das aulas. Para agilizar o atendimento e facilitar o controle das vendas, é importante calcular automaticamente o valor total de cada compra com base na quantidade de cafés solicitados e no preço unitário de cada café.

2- Desenvolva um programa em C que leia a quantidade de cafés comprados por um estudante e o preço unitário de cada café. Em seguida, calcule o valor total da compra e exiba o resultado.

Entrada:

O programa deve solicitar ao usuário a quantidade e o valor unitário.

```
> Qual a quantidade de cafés? 4
> Qual o preço unitário do café? 12.5
```

Saída:

O programa deve exibir o valor total da compra.

```
> O valor total da compra é R$ 50.00
```

Conversões de tipos em C

Conversão implícita: o compilador realiza automaticamente algumas conversões.

Exemplos C - conversao_implicita.c

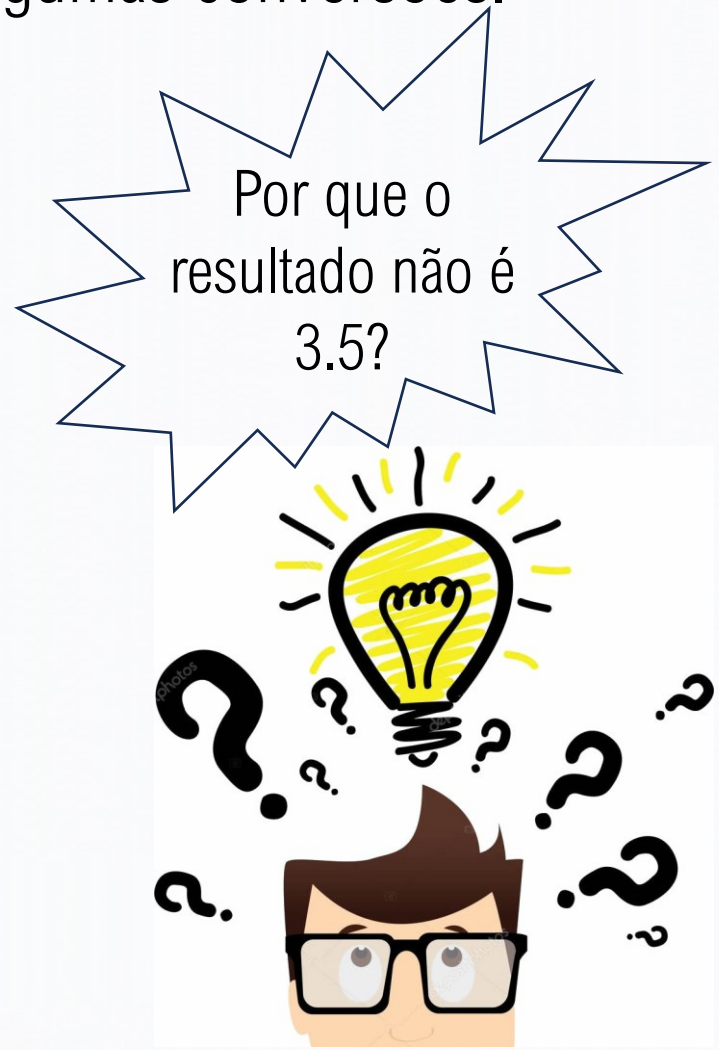
```c
#include <stdio.h>

int main() {
    int a = 5;
    int b = 2;
    float media = (a + b)/2;

    printf("Média: %.2f\n", media);
}
```

Resultado

```
(Prof. Marco 💀 Pichau)-[Exemplos C]
$ ./conversao_implicita
Média: 3.00
```



Conversões de tipos em C

Conversão explícita (casting): o programador força a conversão.

Exemplos C - conversao_explicita.c

```c
#include <stdio.h>

int main() {
    int a = 5;
    int b = 2;
    float media = (float) (a + b)/2;

    printf("Média: %.2f\n", media);
}
```

casting



Resultado

```
(Prof. Marco 💀 Pichau)-[Exemplos C]
$ ./conversao_explicita
Média: 3.50
```

Vamos trabalhar um pouquinho?!!!





Exercícios de aplicação



Em uma oficina mecânica, muitas peças possuem formato circular, como rolamento, engrenagens e polias. Para calcular a quantidade de material necessária para fabricar uma proteção ao redor de uma peça circular, o técnico precisa conhecer o comprimento da borda dessa peça, ou seja, o perímetro da circunferência.

1- Desenvolva um programa em C que leia o valor do raio de uma peça circular (em centímetros), calcule o perímetro da circunferência usando a fórmula: **perímetro = 2 * pi * raio** e exiba o resultado com duas casas decimais.

Entrada:
O programa deve solicitar ao usuário o valor do raio.

```
> Qual a medida do raio (em cm)?
> 5.30
```

Saída:
O programa deve exibir o perímetro calculado.

```
> O perímetro da circunferência
é 33.29 cm
```

Exercícios de aplicação



Uma empresa de paisagismo está elaborando o projeto de um jardim retangular. Para instalar uma cerca ao redor do jardim, é necessário conhecer o seu perímetro, e para calcular a quantidade de grama que será utilizada, é preciso determinar a área do terreno.

2- Desenvolva um programa em C que leia o comprimento e a largura de um jardim retangular (em metros); calcule o perímetro do jardim utilizando a fórmula: **perímetro = 2 * (comprimento + largura)** e a área do jardim utilizando a fórmula: **área = comprimento * largura**. Ao final exiba os resultados com duas casas decimais.

Entrada:

```
> Qual o comprimento do jardim (m)?
> 12.50
> Qual a largura do jardim (m)?
> 8.00
```

Entrada:

```
> Perímetro do jardim: 41.00 m
> Área do jardim: 100.00 m²
```

Exercícios de aplicação

2- Desenvolva um programa em C que leia o comprimento e a largura de um jardim retangular (em metros); calcule o perímetro do jardim utilizando a fórmula: **perímetro = 2 * (comprimento + largura)** e a área do jardim utilizando a fórmula: **área = comprimento * largura.** Ao final exiba os resultados com duas casas decimais.

```
algoritmo retangulo
    real ladoA, ladoB, perim, area
    início
      escreva ("Digite o valor de um lado (em cm): ")
      leia (ladoA)
      escreva ("Digite o valor de outro lado (em cm): ")
      leia (ladoB)
      perim = 2*ladoA  +  2*ladoB
      escreva ("Perímetro: " + perim + " cm ")
      area = ladoA * ladoB
      escreva ("Área do retângulo: " + area + " cm² ")
    fim
```

Exercícios de aplicação



Durante o acompanhamento gestacional, é comum que o tempo de gestação seja informado em semanas. Para fins de estimativa, considere que cada semana possui 7 dias e que um mês possui 30 dias.

3- Elabore um algoritmo e um programa em linguagem C que leia a quantidade de semanas de gestação e calcule a quantidade aproximada de meses correspondentes.

Entrada:

```
> Qual a quantidade de semanas de
gestação?
> 9
```

Entrada:

```
> Tempo de gestação aproximado:
> 2.1 meses
```

Exercícios de aplicação



Em uma Unidade Básica de Saúde, uma equipe deseja estimar o consumo diário de água de um paciente. Para uma orientação inicial, considera-se a recomendação de 35 ml de água por quilograma de massa corporal.

4- Elabore um algoritmo e um programa em C que leia a massa corporal de uma pessoa, em quilogramas, e exiba a quantidade aproximada de água recomendada por dia em litros.

Entrada:

```
> Digite a massa corporal (em kg):
> 70
```

Entrada:

```
> Quantidade aproximada de agua
recomendada por dia: 2.5 litros
```

Exercícios de aplicação



Em um sistema acadêmico, as notas das avaliações são armazenadas com valores reais, permitindo registrar frações de ponto. Para apresentar o desempenho parcial do estudante, é necessário calcular a média aritmética das duas avaliações realizadas.

5- Elabore um algoritmo e um programa em linguagem C que leia as duas notas de um aluno e exiba sua média aritmética, apresentando o resultado com duas casas decimais.

Entrada:

```
> Digite a primeira nota: 7
> Digite a segunda nota: 8
```

Entrada:

```
> Media aritmética: 7.50
```



Alguma dúvida????



Universidade Cruzeiro do Sul

CRÉDITOS

Material elaborado por:

Prof. Marco Antonio Sanches

Material base elaborado por:

- Prof. Alcides
- Prof. Lédon
- Prof. Ana Paula
- Profa. Cristiane

Última atualização: 2026/2

CONTATOS



marco.sanches@cruzeirodosul.edu.br



marco-antonio-sanches-anastácio-601b9123